Contents :

- What is Playwright?
- What is the architecture? How Playwright works in the backend?
- How Playwright works in the front end of the application?
- What are Locators? How does it works?
- What is Wait mechanism? How does it works?

Note : This documentation is based on the Playwright with Java combination

1) What is Playwright?

Playwright is a node.js based API or library which helps to perform web based application and API testing. Please find the below advantages and disadvantages.

Advantages :

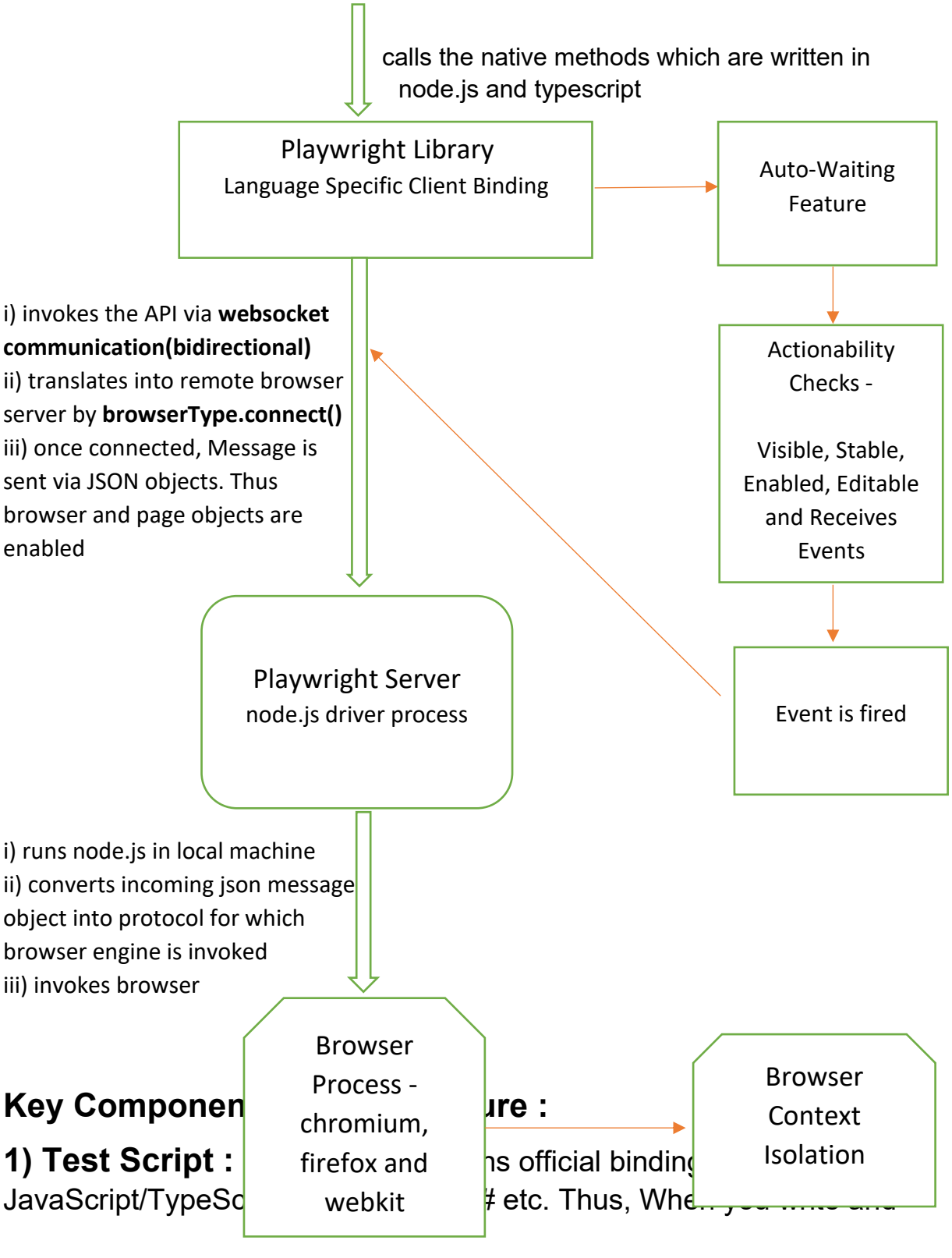
1) Playwright is a open source automation library
2) It supports cross browser, cross platform and cross programming language
3) It supports auto-waiting capability
4) It has inbuilt web based assertions and trusted events
5) It supports tracing which helps to get test retry strategy, capture execution trace, videos and screenshots
6) It automatically supports Iframe and Shadow DOM elements
7) It has inbuilt codegen which generates the test script in the chosen programming language based on the live browser actions
8) It has inbuilt inspector which helps for debugging and inspect page, generate selectors and assert visibility

Disadvantages :

1) Playwright does not support mobile based application testing like Android.apk file or IOS.ipa file. It supports mobile web emulation
2) It does not support database testing. It does not have inbuilt database driver. It has to rely on third party API

2) What is the architecture? How Playwright works in the backend?





Key Componen ure :

1) Test Script :

ns official bindin

JavaScript/TypeSc # etc. Thus, Whe

execute the test script, The inbuilt client binding translates your code into the ongoing protocol message and invokes the native methods (which are already written in node.js and typescript) and these methods are sent via websocket communication protocol to the playwright server. When it comes to programming language, Playwright with JavaScript and TypeScript ecosystem provides advantage like inbuilt fixtures and vs code extension. Although Java also provides inbuilt fixtures by integrating Junit. Apart from this we also have Python fixtures. Every programming language has its own advantage.

I would like to explain the fundamentals of the below mentioned piece of code which helps to run the test script



Playwright : Adds environment variables that will be passed to the driver class
Browser : Running instance of chromium, firefox and webkit
BrowserContext : An isolated session within the browser
Page : A single tab or pop up inside a context. The actual object which helps to interact with the web application

.create() : The method is derived from the Playwright interface.
i) creates and install browser binaries
ii) creates driver instance and install drivers with the help of environment and installed browsers parameters
iii) processBuilder – launch node.js driver as an OS process. It is the first handshake message protocol
iv) initialize driver : It sends “initialize” message to the Node.js driver over the connection and blocks which are waiting for JSON response. This is the moment when Playwright Library connects with the Playwright Server.

.launch() : launch method comes from the Browser interface. It launches browser that uses storage which is located at userDataDir (which stores

cookies and local storage) and returns the context. Closing this context will close the browser.
.newContext() : It is the mechanism which is coming from BrowserContext interface. It runs the already running browser process but gives you completely fresh, incognito like session, separate cookies, local storage and cache. With the help of this we can also perform safe parallel test execution
.newPage() : Every action methods like page.locator(#a).click() and page.locator(#b).fill() is called from “Page” interface which sends the message through websocket protocol to the Playwright server and on to the actual browser.

2) Playwright Library : It helps to invoke the API via websocket communication protocol which is bidirectional. Websocket protocol stays connected for the entire session. It uses Stdio pipe between your test process and driver process to communicate on the same machine. The backend system is not like selenium webdriver W3C standardized protocol where the commands are sent through the HTTP request and it gets back the response which means whenever a test script is repeatedly executed, It creates a new session and HTTP request is sent over again and it gets back the response. Websocket communication protocol does not creates a new session everytime. Thus, Browser Context Isolation mechanism comes into the picture. The main motive of Playwright Library is to translate the native methods or API into the remote browser server with the help of pre defined method “browserType.Connect()”. It also has another pre defined method “browserType.ConnectOverCDP()” which converts directly to the chrome dev tools protocol rather than following playwright server which runs the chrome/chromium instance. Once they are connected, Message is sent via JSON objects. Thus, Browser and Page objects are enabled.
Apart from this, It also has Auto-Waiting feature which checks for the elements to be visible, stable, enabled, editable, received events in the document object model (DOM) and then the events are fired. Thus, The message is forwarded to the websocket communication protocol.
The advantage of Auto-Waiting feature is to avoid flaky tests.

3) Playwright Server : When pre defined method “playwright.chromium.launch()” is executed, Playwright Server runs

node.js driver as a child process on the local machine and it stays alive for the duration of your test session and turns down when the test script closes browser. The main motive of Playwright Server is to convert incoming JSON message from client into the protocol for which the browser engine is involved.

Browser Engines :
Chromium → Chrome Dev Tools protocol which helps to invoke chromium browser instance
Firefox → A custom automation protocol which converts into playwright patched firefox build and invoke firefox browser instance
Webkit → A custom automation protocol which converts into playwright patched webkit build and invoke webkit browser instance

4) How does Playwright Server and Browser Server actually works?

When Playwright Browser Server is started (npx playwright launch server), It launches a real browser and exposes a websocket endpoint for it. Your test script calls “browserType.Connect()” by passing that websocket URL. Thus, It connects client to the remote browser server or driver. Once connected, It invokes the browser and uses Browser and Page objects the same way as locally are launched (browser.newPage() and page.locator(“#a”).click()). When browser is launched, It has the ability to isolate the session where it has fresh cookie and local storage

3) How Playwright works in the front end of the application?

Playwright works on the event based driven and uses a browser Chrome Dev Tools protocol which enables smarter auto-waiting feature to launch

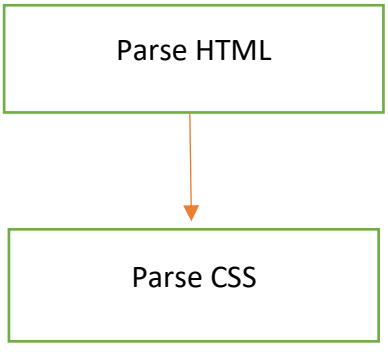
chrome/chromium instance. As mentioned, It uses test script which calls Playwright Library and connects to the Playwright Server to invoke firefox and webkit browser instance.

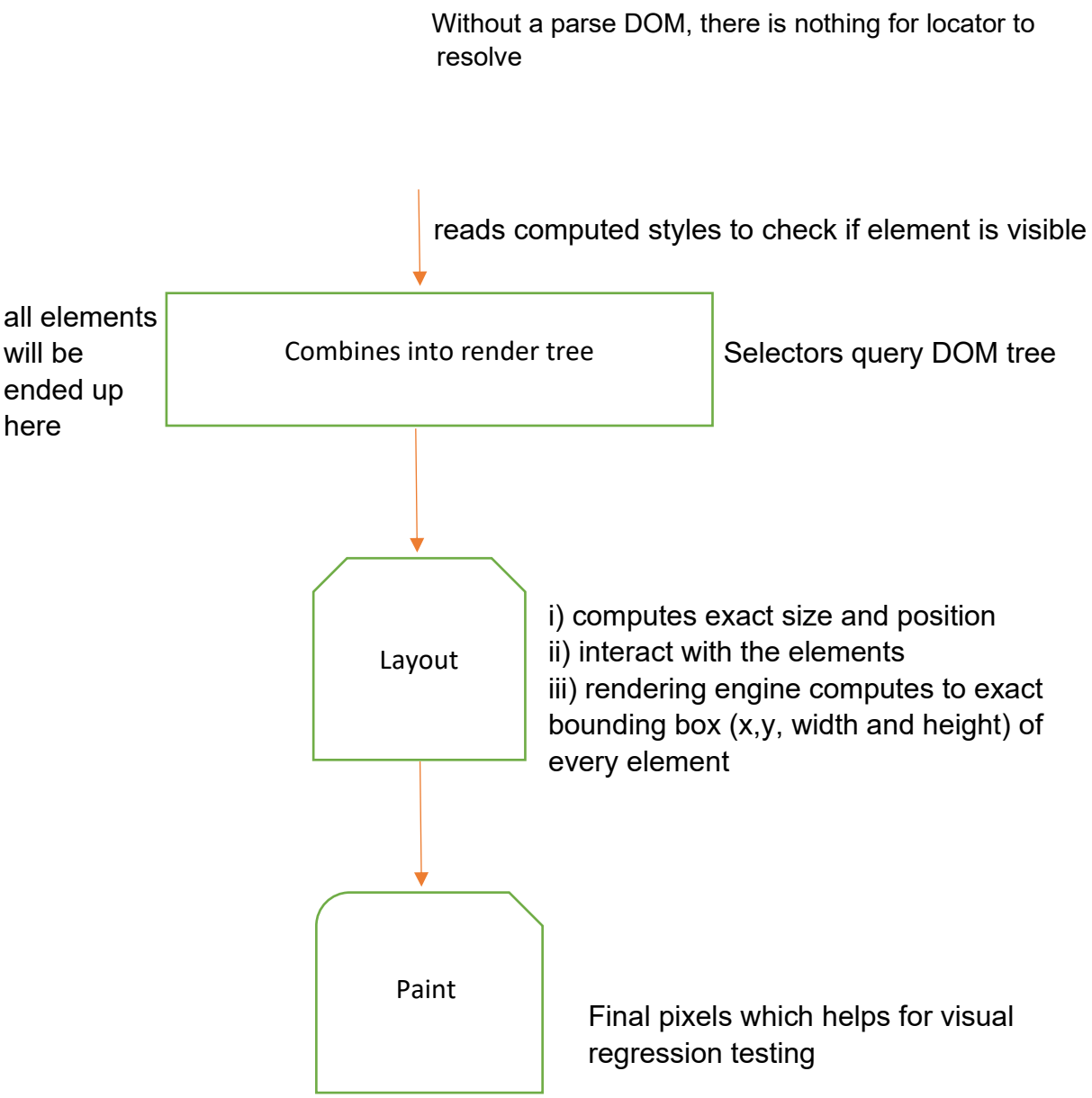
How does it works?
It waits for i) loads the events
ii) DOM content is loaded or refreshed
iii) Network is idle (No pending request)
iv) UI to be stable

It supports single page application (SPA) where the
i) HTML page loads initially
ii) Navigation happens without refresh
iii) UI updates via JavaScript framework and AJAX calls

It supports **rendering engines** (translates SPA into webpage)
Rendering engine is a browser engine where the core software component is inside every browser that takes the raw HTML, CSS, JavaScript and turns them into pixels which you can see in the screen.

PTO :



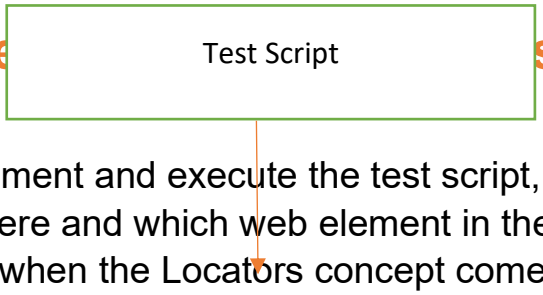


Playwright follows Page LifeCycle Events which helps to understand how a page loads and how to synchronize automation with browser activity.

i) domContentLoaded : Triggered when initial HTML is parsed and DOM is ready
ii) load : All resources are fully loaded (CSS, JavaScript)

iii) networkidle : Triggered when there are no network calls which means there are no pending requests
iv) commit : Browser has committed to navigation

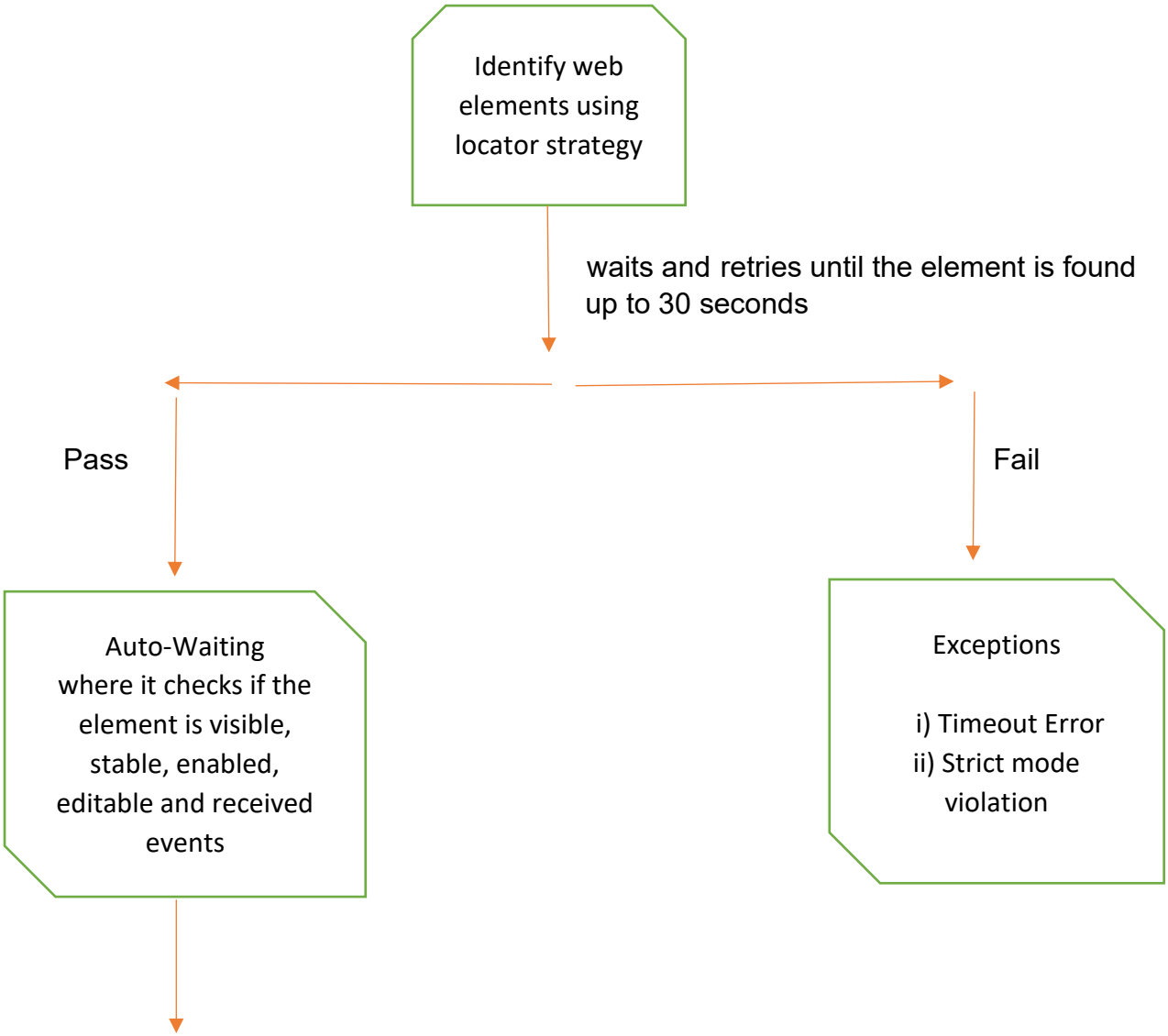
4) What are s it works?



When you implement and execute the test script, How does your test script knows where and which web element in the webpage should interact? That's when the Locators concept comes into the picture. Locators are the in built methods which helps to interact with the web elements in the application.
Specifically Locator is the interface which has default maximum timeout set up to 30 seconds. The default value can be changed by using BrowserContect.setDefaultTimeOut() and Page.SetDefaultTimeOut() methods. Pass 0 to disable timeout.

How does Locators works?
When your test script calls locators, It tries to locate the web element which is present in the HTML DOM structure and retries until the default maximum timeout which is set to 30 seconds. Then it follows auto-waiting feature where it tries to check if the web element is visible, stable, enabled, editable and events received. If found then it interacts with the web element. It throws exception if it fails to find the web element. 2 exceptions are found. They are
i) TimeoutError – This error occurs when the test script using locator strategy exceeds 30 sec and still retries to find the web element
ii) Strict mode violation – This error occurs when the test script using locator strategy tries to find more than 1 web element in the DOM with the same attributes and values

calls locators



PTO :

Below are the list of playwright locators which are best known to interact with the web elements :

1) page.getByText() : The method is best to use for the available links

Syntax : Locator link = page.getByText("Forgot your password?");

```
public void verifyLink()  1 usage  Playwright Local
{
    Locator link = page.getByText("Forgot your password?");
    PlaywrightAssertions.assertThat(link).isVisible();
    link.click();
}
```

Parameter : It takes text as a parameter and locates the element

2) page.getByLabel() : The method is best to use for the available radio button, checkbox and dropdown

Syntax : Locator agreeCheckbox = page.getByLabel(" I agree to the ");

```
public void verifyCheckbox2()  1 usage  Playwright Local
{
    Locator agreeCheckbox = page.getByLabel( text: " I agree to the ");
    if(agreeCheckbox.isEditable())
    {
        agreeCheckbox.click();
        PlaywrightAssertions.assertThat(agreeCheckbox).isChecked();
    }
}
```

Parameter : It takes the value as a parameter which are associated with <label>, <aria-labelledby> and <aria-label> and locates the element

PTO :

3) page.getByPlaceholder() : The method is best to use for the available edit box (username and password). It locates the input elements

Syntax : Locator password = page.getByPlaceholder("Password");

```
public void verifyPassword(String value)  1 usage  Playwright Local
{
    Locator password = page.getByPlaceholder( text: "Password");
    if(password.isEnabled() && password.isEditable())
    {
        password.waitFor();
        password.fill(value);
    }
}
```

Parameter : It takes the placeholder attribute value with the input tagname as a parameter and locates the element

4) page.getByAltText() : The method is best to use for the available image logo.

Syntax : Locator password = page.getByAltText("Password");

```
public void verifyProduct()  no usages  new *
{
    Locator product1 = page.getByAltText("Product Image");
    PlaywrightAssertions.assertThat(product1).isVisible();
    product1.click();
}
```

Parameter : It takes the alt text attribute value as a parameter and locates the element

5) page.getByTitle() : The method is best to use for the available title attributes

Syntax : Locator productTitle = page.getByTitle("Product Title");

```
public void verifyProduct()  no usages  new *
{
    Locator productTitle = page.getByTitle( text: "Product Title");
    PlaywrightAssertions.assertThat(productTitle).hasText( expected: "25 issues");
}
```

Parameter : It takes the title attribute value as a parameter and locates the element

6) page.getByTestId() : The method is best to use when there is dynamic content, redesigned UI and data-testid attributes

Syntax : Locator product = page.getByTestId("product-title");

```
public void verifyProduct()  no usages  new *
{
    Locator product = page.getByTestId("product-title");
    PlaywrightAssertions.assertThat(product).isVisible();
}
```

Parameter : It takes the data-testid attribute as a parameter and locates the element

7) page.getByRole() : The method is best to use for the available buttons, links, checkboxes, headings, textboxes and menu items. It is used when the elements have meaningful accessibility role and accessible name

Syntax : page.getByRole(AriaRole.BUTTON, new Page.getByRoleOptions().setName("value").click();

```
78
79      public void verifyButton()  1 usage  Playwright Local
80      {
81          Locator button = page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign In"));
82          PlaywrightAssertions.assertThat(button).isVisible();
83          button.click();
84      }
85
```

Parameter : It takes data-testid and available attributes as a parameter and locates the element

PTO :

8) Locator with CSS (Cascading Style Sheet) : CSS are used for speed, readability and stability with the attributes. CSS can traverse from parent to child. The advantage of CSS is that it can interact shadom dom web elements.

Syntax : Locator username = page.locator("css=input#user-name");
Locator password = page.locator("input#password");

```
public void verifyCSSLocator()  no usages  new *
{
    Locator username = page.locator( selector: "css=input#user-name");
    username.fill( value: "standard_user");
    Locator password = page.locator( selector: "input#password");
    password.fill( value: "secret_sauce");
}
```

Parameter : It takes css syntax value as a parameter and locates the element

9) Locator with Xpath (XML Path Language) : Xpath are used when the HTML DOM is complex, when we are trying to build a complex implementation. We can also parameterize the xpath. It can traverse in any direction. The advantage of Xpath and Xpath Axes is that it can interact dynamic web elements.

Syntax : Locator username = page.locator("xpath=//input[@id='user-name']");
Locator username = page.locator("//input[@id='user-name']");

```
public void Login(String user) {  no usages  new *

    if(page.locator( selector: "//input[@id='user-name' and @class='input_error form_input']").isVisible())
    {
        page.locator( selector: "//input[@id='user-name' and @class='input_error form_input']").fill(user);
    }
}
```

Parameter : It takes xpath syntax value as a parameter and locates the element

PTO :

10) : Locator with frame : iframe is a separate HTML DOM which is present inside the parent HTML DOM structure. Frame is nothing but the web elements which are present inside iframe HTML DOM structure. When you execute with the actual action methods, Then it tries to find the element which is not possible because the iframe

web elements are not visible and it throws TimeoutError. When you execute the exact syntax, It uses pre defined method .frameLocator() (derived from FrameLocator interface) which has in built array of all frames attached to the webpage. Thus, It interacts with the frame web elements.

Syntax : FrameLocator frame = page.frameLocator("iframe#payment-frame");

```
public void verifyFrame(String user)  no usages  new *
{
    FrameLocator frame = page.frameLocator( selector: "iframe#payment-frame");
    frame.locator( selectorOrLocator: "input[name='cardNumber']").fill( value: "4242424242424242");
}
```

Parameter : It takes attribute value with the tag name iframe and frame as a parameter and locates the element

11) Locator with Shadow DOM : Shadow DOM is a hidden mini DOM element which is attached to the parent DOM structure. Shadow DOM element starts with the tagname “#shadow-root”. These elements are not visible. If you try to execute with the actual action method then it throws TimeoutError. Thus, It cannot be directly interacted with the locators. When you use css locator strategy with the help of Locator interface, It actually looks for the shadow-root tagname and pierce through it and interacts with it. This is a in built mechanism

Syntax : Locator username = page.locator("custom-login-widget input#username");

```
public void verifyFrame(String user)  no usages  new *
{
    // Assume <custom-login-widget> has an open shadow root containing <input id="username">
    Locator username = page.locator( selector: "custom-login-widget input#username");
    username.fill( value: "standard_user");
}
```

Parameter : It looks for shadow-root (open) tagname in the HTML DOM structure, takes the CSS attribute value as a parameter and locates the element

PTO :

5) What is Wait mechanism? How does it works?

Wait mechanism plays a major role in terms of test execution. The advantage of wait mechanism is to avoids flaky tests and make the test execution smoother. The main motive is to perform end to end testing without any issues. When we are working on any web elements like radio button or checkbox then auto-waiting feature comes into the picture. Sometimes there might be dynamic web elements, delay, server issues or application issues. To overcome this problem, Custom wait which is also known as explicit wait plays a major role. Now we have lot of explicit wait options. These options gives us the flexibility to check if the web element state is attached, detached, hidden and visible. Sometimes when we are working on test execution, Our test scripts execution is faster than the application response. This problem is known as Synchronization which is the most challenging part. The issue might be related to backend configuration or application environment issue. This causes flaky tests which are harder to maintain. It can be fixed by auto-waiting feature, network based wait and explicit wait.

PTO :